

claude-windows / 2ba49afa-99f4-41f2-86a2-27a289d432f3

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\2ba49afa-99f4-41f2-86a2-27a289d432f3\subagents\workflows\wf_d954e9ee-fd0\agent-add2a424a24678899.jsonl`
- SHA-256: `db52dee00c4e663abdc6af366d74a096e25f042c5d23efd5b71c3c613f5b915`
- Source modified: `2026-06-21T15:16:59+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_d954e9ee-fd0`
- session_id: `2ba49afa-99f4-41f2-86a2-27a289d432f3`
```

Transcript

1. user (2026-06-21T15:13:32.456Z)

You are reviewing for REGRESSIONS and ORPHANED i18n keys. A5 refactor under C:/Users/avidu/Projects/khelsutra/web/src: a shared error->text mapping.

- NEW lib/errorText.ts: errorText(t, e) ? engine code -> e.detail ?? t("errors.engineGeneric"); else t(`errors.\${e.code}`).
- App.tsx onBuild catch now: classifyError(err) -> errorText(t, e) (was: parseDetail(err.body) ?? String(err)). parseDetail import removed.
- IngestPanel.tsx: errorMsg now = errorText(t, phase.error) (was: inline ingest.\${code} map).
- Catalogs (en/kn/hi): MOVED ingest.{networkError,invalidUri,emptyResult,timeout} -> new errors.* namespace; ADDED errors.engineGeneric; REMOVED ingest.failed. ingest.* kept: title/intro/uriPlaceholder/uriAriaLabel/hint/submit/submitting/cancel/working/progress/retry.

Tests: 135 green incl. errorText.test.ts, build-error.test.tsx, the existing IngestPanel.test.tsx, i18n key-parity.

Do these checks against the files on disk:

1. ORPHANED KEYS: grep the ENTIRE C:/Users/avidu/Projects/khelsutra/web/src tree for any lingering reference to the REMOVED keys: t("ingest.failed"), t("ingest.networkError"), t("ingest.invalidUri"), t("ingest.emptyResult"), t("ingest.timeout"), or template t(`ingest.\${...}`). Any remaining reference is a BUG (would render a raw key). Also confirm nothing references a key that no longer exists.
 2. KEY-PARITY: confirm errors.{networkError,invalidUri,emptyResult,timeout,engineGeneric} exist in ALL THREE catalogs (en/kn/hi) and that the removed ingest keys are gone from all three (so the hygiene key-parity test stays green). List any asymmetry.
 3. BUILD MAPPING CORRECTNESS: read lib/errors.ts classifyError + App.tsx onBuild. Confirm: a bare 5xx -> networkError -> friendly line; a 4xx/anything WITH a {detail} -> engine -> verbatim detail; a fetch throw -> networkError. Flag any case where the build path would now show a WORSE message than before (e.g. losing a real engine detail).
 4. INGEST TEST TEXT: read components/IngestPanel.test.tsx ? confirm the asserted strings ("can't reach khelsutra", the 400 detail) are still produced by errorText + the new errors.* copy (same rendered text).
- Be precise (file:line). Empty findings if clean.

2. assistant (2026-06-21T15:13:36.625Z)

I'll review the A5 refactor for regressions and orphaned i18n keys. Let me start by examining the relevant files.

3. user (2026-06-21T15:13:37.508Z)

```
1      // The ONE place a classified error (lib/errors.ts) becomes user-facing text. Both
surfaces use it:
2      // the ingest flow (IngestPanel) and the build flow (App.onBuild ? CompileResult), so a
5xx / engine-
3      // down failure reads the same friendly way everywhere instead of a raw "HTTP 500" /
fetch message.
```

```

4      //
5      // `engine` carries the engine's OWN message (the most specific) ? shown verbatim, never
translated.
6      // Every other code maps to a friendly localized line in the shared `errors.*`
namespace. Kept out of
7      // errors.ts so that module stays i18n-free and unit-testable without a translation
context.
8      import type { TFunction } from "i18next";
9      import type { IngestError } from "./errors";
10
11     export function errorText(t: TFunction, e: IngestError): string {
12         if (e.code === "engine") return e.detail ?? t("errors.engineGeneric");
13         return t(`errors.${e.code}`);
14     }
15

```

4. user (2026-06-21T15:13:37.982Z)

```

1      // Error helpers shared by the ingest flow and the build flow. Kept i18n-FREE: callers
map an
2      // IngestErrorCode to a localized string (so hooks stay testable without a translation
context),
3      // while the engine's own `detail` text passes through verbatim (it's the most specific
message).
4      import { ApiError } from "../api/client";
5
6      /** Pull FastAPI's {"detail": "..."} message out of an error body, else return the raw
body. */
7      export function parseDetail(body: string): string {
8          try {
9              const j = JSON.parse(body) as { detail?: unknown };
10             if (j && typeof j.detail === "string") return j.detail;
11         } catch {
12             /* not JSON ? fall through to the raw body */
13         }
14         return body;
15     }
16
17     /** Stable, localizable failure categories surfaced by the ingest flow.
18     * `engine` carries the engine's own message in `detail`; the rest are localized by
code. */
19     export type IngestErrorCode = "invalidUri" | "networkError" | "emptyResult" | "timeout"
| "engine";
20
21     export interface IngestError {
22         code: IngestErrorCode;
23         /** Present for `engine` ? the verbatim FastAPI detail / status; rendered as-is, not
translated. */
24         detail?: string;
25     }
26
27     /** Classify a thrown client error into a stable code. A network throw (ApiError with no
`status`)
28     * is the common "engine unreachable" case ? its own friendly code; any HTTP error
surfaces the
29     * engine's `detail` (e.g. an unsupported-scheme 400, a not-found 404). */
30     export function classifyError(err: unknown): IngestError {
31         if (err instanceof ApiError) {
32             if (err.status === undefined) return { code: "networkError" }; // fetch threw ?
engine truly unreachable
33             if (err.body) {
34                 const detail = parseDetail(err.body);
35                 if (detail) return { code: "engine", detail }; // a real engine message (e.g. a
400 unsupported-scheme)
36             }

```

```

37      // A bare 5xx with no body is almost always the engine being down / a gateway that
can't reach it
38      // (e.g. the dev proxy when :8000 is off) ? for a non-technical user a friendly "is
it running?"
39      // beats a cryptic "HTTP 500". A 4xx with no body keeps the status (it's a client-
side mistake).
40      if (err.status >= 500) return { code: "networkError" };
41      return { code: "engine", detail: `HTTP ${err.status}` };
42  }
43  return { code: "engine", detail: String(err) };
44  }
45

```

5. user (2026-06-21T15:13:38.919Z)

```

1      // The front-half ingest screen (appliance S1/S2, tracker B-P2). Point the tool at a
video ? a
2      // `gs://` bucket URI or a path on the engine host ? and it submits to POST
/api/process, shows
3      // live progress while polling GET /api/jobs/{id}, and hands the resulting video_id to
the existing
4      // rally-selection back-half (App lifts ?video=<id>). Honest by construction: the engine
reports
5      // coarse phase WORDS (not a %), and a success-shaped failure (done but no rallies) is
shown, never
6      // silently treated as a load.
7      import { useState } from "react";
8      import type { FormEvent } from "react";
9      import { useTranslation } from "react-i18next";
10     import { useIngestJob } from "../hooks/useIngestJob";
11     import { errorText } from "../lib/errorText";
12
13     interface Props {
14         /** Called with the engine's video_id once a submitted job finishes successfully. */
15         onLoaded: (videoId: string) => void;
16     }
17
18     export function IngestPanel({ onLoaded }: Props) {
19         const { t } = useTranslation();
20         const [uri, setUri] = useState("");
21         const { phase, submit, cancel, reset, busy } = useIngestJob(onLoaded);
22
23         const onSubmit = (e: FormEvent) => {
24             e.preventDefault();
25             submit(uri);
26         };
27
28         // Shared mapping with the build flow (lib/errorText): `engine` failures carry the
engine's own
29         // message (a 400 scheme/path detail, a worker error); every other code maps to a
friendly
30         // localized line ? the network case especially, since this is the first screen a
fresh user hits
31         // and "is the engine running?" must read plainly.
32         const errorMsg = phase.status === "error" ? errorText(t, phase.error) : null;
33
34         return (
35             <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
36                 <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
37                 <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
38
39                 <form onSubmit={onSubmit} className="mt-3 flex flex-wrap items-center gap-2">
40                     <input
41                         type="text"

```

```

42         value={uri}
43         onChange={(e) => setUri(e.target.value)}
44         placeholder={t("ingest.uriPlaceholder")}
45         aria-label={t("ingest.uriAriaLabel")}
46         disabled={busy}
47         className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15 bg-
white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
48     />
49     <button
50         type="submit"
51         disabled={busy}
52         className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
53     >
54         {phase.status === "submitting" ? t("ingest.submitting") : t("ingest.submit")}
55     </button>
56     {busy && (
57         <button
58             type="button"
59             onClick={cancel}
60             className="min-h-11 rounded-xl border border-black/15 px-4 py-2 text-sm
hover:border-green"
61         >
62             {t("ingest.cancel")}
63         </button>
64     )}
65 </form>
66
67 <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
68
69     {/* Live status (aria-live so screen readers hear progress). Indeterminate: the
engine reports
70         phase words ("hashing", "segmenting"), not a percentage ? so a spinner + the
phase word. */}
71     <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
72         {busy && (
73             <p className="flex items-center gap-2 text-ink/70">
74                 <Spinner />
75                 <span>
76                     {t("ingest.working")}
77                     {phase.status === "polling" && phase.progress
78                         ? ` · ${t("ingest.progress", { progress: phase.progress })}`
79                         : ""}
80                 </span>
81             </p>
82         )}
83         {phase.status === "error" && (
84             <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
85                 <p>{errorMsg}</p>
86                 <button
87                     type="button"
88                     onClick={reset}
89                     className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
90                 >
91                     {t("ingest.retry")}
92                 </button>
93             </div>
94         )}
95     </div>
96 </section>
97 );
98 }
99

```

```

100     function Spinner() {
101         return (
102             <span
103                 className="inline-block h-3.5 w-3.5 animate-spin rounded-full border-2 border-
ink/20 border-t-ink/70"
104                 aria-hidden="true"
105             />
106         );
107     }
108

```

6. user (2026-06-21T15:13:39.007Z)

```

1      // @vitest-environment jsdom
2      //
3      // The ingest screen end to end: type a URI ? submit ? (mocked engine) ? onLoaded gets
the video_id;
4      // and the two failure surfaces a non-technical user actually hits ? the engine's own
400 detail,
5      // and a friendly "is the engine running?" when it's unreachable. Mounts the real i18n
catalog.
6      import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
7      import { render, screen, fireEvent, cleanup, act } from "@testing-library/react";
8      import i18n from "../i18n/index";
9      import { IngestPanel } from "../IngestPanel";
10     import type { JobStatus } from "../api/types";
11
12     type FetchFn = typeof fetch;
13
14     function httpOk(status: number, json: unknown) {
15         return { ok: status >= 200 && status < 300, status, json: async () => json, text:
async () => JSON.stringify(json) };
16     }
17
18     function fetchSeq(process: unknown, jobs: JobStatus[]): FetchFn {
19         let i = 0;
20         return vi.fn(async (url: string | URL, init?: RequestInit) => {
21             const u = String(url);
22             if (u === "/api/process" && (init?.method ?? "GET") === "POST") return process;
23             if (u.startsWith("/api/jobs/")) {
24                 const body = jobs[Math.min(i, jobs.length - 1)];
25                 i += 1;
26                 return httpOk(200, body);
27             }
28             throw new Error(`unexpected fetch ${u}`);
29         }) as unknown as FetchFn;
30     }
31
32     beforeEach(async () => {
33         window.localStorage.clear();
34         await i18n.changeLanguage("en"); // guarantees init completed + a known locale for the
assertions
35     });
36
37     afterEach(() => {
38         cleanup();
39         vi.restoreAllMocks();
40         vi.unstubAllGlobals();
41         vi.useRealTimers();
42     });
43
44     const typeUri = (value: string) =>
45         fireEvent.change(screen.getByLabelText(/video location/i), { target: { value } });
46     const clickProcess = () => fireEvent.click(screen.getByRole("button", { name: /process
video/i }));

```

```

47
48 describe("IngestPanel", () => {
49   it("submits a URI, polls, and hands the video_id to onLoaded", async () => {
50     vi.useFakeTimers();
51     vi.stubGlobal(
52       "fetch",
53       fetchSeq(httpOk(202, { job_id: "j", status: "queued", poll: "/api/jobs/j" })), [
54         { job_id: "j", status: "running", progress: "segmenting" },
55         { job_id: "j", status: "done", video_id: "vidZ", result: { status: "success",
video_id: "vidZ" } }],
56       ],
57     );
58     const onLoaded = vi.fn();
59     render(<IngestPanel onLoaded={onLoaded} />);
60     typeUri("gs://bucket/match.mp4");
61     clickProcess();
62
63     await act(async () => {
64       await vi.advanceTimersByTimeAsync(0); // flush submit ? start polling
65     });
66     await act(async () => {
67       await vi.advanceTimersByTimeAsync(1000); // running tick
68     });
69     await act(async () => {
70       await vi.advanceTimersByTimeAsync(1500); // done tick
71     });
72     expect(onLoaded).toHaveBeenCalledWith("vidZ");
73   });
74
75   it("shows the engine's own 400 detail (e.g. an unsupported scheme)", async () => {
76     vi.stubGlobal("fetch", fetchSeq(httpOk(400, { detail: "unsupported storage URI
scheme: ftp://" })), []));
77     render(<IngestPanel onLoaded={vi.fn()} />);
78     typeUri("ftp://nope");
79     clickProcess();
80     expect(await screen.findByText(/unsupported storage URI scheme/i)).toBeTruthy();
81   });
82
83   it("shows a friendly, localized message when the engine is unreachable", async () => {
84     vi.stubGlobal("fetch", vi.fn(async () => {
85       throw new Error("offline");
86     }) as unknown as FetchFn);
87     render(<IngestPanel onLoaded={vi.fn()} />);
88     typeUri("gs://bucket/match.mp4");
89     clickProcess();
90     expect(await screen.findByText(/can't reach khelsutra/i)).toBeTruthy();
91   });
92
93   it("rejects an empty URI with a hint and never calls the engine", () => {
94     const f = vi.fn();
95     vi.stubGlobal("fetch", f);
96     render(<IngestPanel onLoaded={vi.fn()} />);
97     clickProcess();
98     expect(screen.getByTestId("ingest-error")).toBeTruthy();
99     expect(f).not.toHaveBeenCalled();
100   });
101 });
102

```

7. assistant (2026-06-21T15:13:44.507Z)

Now let me check the catalogs and grep for orphaned keys.

8. user (2026-06-21T15:13:46.553Z)

No matches found

9. user (2026-06-21T15:13:47.502Z)

No matches found

10. user (2026-06-21T15:13:47.807Z)

```
web\src\i18n\i18n.test.tsx
web\src\i18n\index.ts
web\src\i18n\hygiene.test.ts
web\src\i18n\locales\en\common.json
web\src\i18n\locales\kn\common.json
web\src\i18n\locales\hi\common.json
```

11. assistant (2026-06-21T15:13:50.793Z)

Grep finds no orphaned references. Now let me read the three catalogs and the build-error test plus App.tsx onBuild.

12. user (2026-06-21T15:13:51.280Z)

```
1      {
2        "app": {
3          "badge": "pick your rallies · alpha",
4          "title": "Pick the rallies for your reel",
5          "intro": "Filter by <b>rally length</b> ? e.g. <i>?top 15 longest?</i> ? then fine-
6            tune by tapping cards. Everything starts <b>selected</b>; just drop the ones you don't want."
7        },
8        "lang": {
9          "label": "Language",
10         "en": "English",
11         "kn": "?????",
12         "hi": "?????"
13       },
14       "bulk": {
15         "selectAll": "Select all",
16         "clear": "Clear",
17         "invert": "Invert"
18       },
19       "mode": {
20         "demo": "These are sample rallies ? process a video above to work on a real match.",
21         "loading": "Finding the rallies in your video?",
22         "error": "Couldn't load this video's rallies: {error}",
23         "loaded": "Found <b>{count}</b> rallies in your video."
24       },
25       "query": {
26         "placeholder": "e.g. over 10s · top 15 longest · shortest first",
27         "ariaLabel": "Filter rallies by length, count or order",
28         "apply": "Apply",
29         "notAvailable": "not available yet",
30         "notAvailableTitle": "?{token}? needs {kind} detection ? not available yet",
31         "notAvailableHelp": "Shot type, players and score aren't available yet ? filter by
32           rally length instead.",
33         "unreadable": "Couldn't read that ? try ?over 10s? or ?top 15 longest?.",
34         "hint": "Type over 10s for rallies longer than 10 seconds ? or tap a sample below.",
35         "try": "Sample queries:",
36         "support": "Filtering by <b>rally length</b> is fully supported. Shot-count is
37           <x>experimental</x> (under development)."
38       },
39       "grid": {
40         "empty": "No rallies to show yet.",
41         "rally": "Rally {n}",
42         "inReel": "Rally {n}, in reel",
```

```

40     "notInReel": "Rally {n}, not in reel",
41     "start": "start",
42     "long": "long",
43     "shots": "~{count} shots · experimental",
44     "shotsTitle": "Shot count is experimental (under development) and may be
inaccurate{conf}. Filter by rally length instead.",
45     "shotsConfidence": " ? confidence: {conf}"
46 },
47     "footer": {
48         "count": "{count, plural, one {# rally} other {# rallies}}",
49         "summaryRest": "selected · {total} total",
50         "reelName": "Reel name",
51         "reelNamePlaceholder": "my-highlights",
52         "build": "Build reel ({count})",
53         "building": "Building?",
54         "floorOne": "<b>Your only selected rally</b> has fewer than {min} shots ? it would
be skipped, so the build would fail. Pick a longer rally.",
55         "floorAll": "<b>All {count} selected rallies</b> have fewer than {min} shots ?
they'd all be skipped, so the build would fail. Pick longer rallies.",
56         "floorSomeLead": "{count, plural, one {# rally} other {# rallies}}",
57         "floorSomeRest": "with fewer than {min} shots will be skipped when building (they're
too short).",
58     },
59     "result": {
60         "ready": "Your reel is ready",
61         "failed": "Build failed",
62         "dismiss": "Dismiss",
63         "download": "Download {filename}"
64     },
65     "build": {
66         "noVideo": "These are sample rallies ? process a real video first to build a reel."
67     },
68     "ingest": {
69         "title": "Process a video",
70         "intro": "Point KhelSutra at a video and it finds the rallies for you ? this can
take a few minutes.",
71         "uriPlaceholder": "gs://your-bucket/match.mp4",
72         "uriAriaLabel": "Video location ? a gs:// bucket URL or a path on the engine host",
73         "hint": "Paste a cloud-storage link (gs://?), or a path to a video on the machine
running KhelSutra.",
74         "submit": "Process video",
75         "submitting": "Submitting?",
76         "cancel": "Cancel",
77         "working": "Processing your video?",
78         "progress": "{progress}",
79         "retry": "Try again"
80     },
81     "errors": {
82         "networkError": "Can't reach KhelSutra. Is it running?",
83         "invalidUri": "Enter a video location first ? a gs:// link or a path on the machine
running KhelSutra.",
84         "emptyResult": "Processing finished but found no rallies in this video. Try a
clearer match recording.",
85         "timeout": "This is taking longer than expected. The job may still be running ?
reload with the video link, or try again.",
86         "engineGeneric": "Something went wrong. Please try again."
87     },
88     "selection": {
89         "count": "{count, plural, one {# rally selected} other {# rallies selected}}"
90     }
91 }
92

```



```
1      {
2      "app": {
3          "badge": "?????-????? ???? . ????",
4          "title": "????? ?????????? ?????????????? ????",
5          "intro": "<b>????? ????</b>? ?????????? ?????? ? ????. <i>????? ?????? ???
15?</i> ? ???? ?????????????? ?????? ???? ?????????????? ??????????. ?????? <b>?????????</b>
????????????????; ?????????????? ?????? ??????????."
6      },
7      "lang": {
8          "label": "?????",
9          "en": "English",
10         "kn": "?????",
11         "hi": "?????"
12     },
13     "bulk": {
14         "selectAll": "????????? ??????????",
15         "clear": "?????????????",
16         "invert": "?????????"
17     },
18     "mode": {
19         "demo": "????? ??? ? ?????? ?????? ???? ?????? ?????????????? ??????????????
<code>?video=?video_id?</code> ???? ??????????",
20         "loading": "<code>{videoId}</code> ???? ?????????????? ???? ??????????????????",
21         "error": "<code>{videoId}</code> ???? ?????????????? ???? ??????????????: {error}",
22         "loaded": "? ? ?????????? <b>{count}</b> ?????????????? ???? ??????????".
23     },
24     "query": {
25         "placeholder": "???. over 10s · top 15 longest · shortest first",
26         "ariaLabel": "????, ????? ???? ?????? ?????????? ?????????????? ??????? ????",
27         "apply": "?????????",
28         "notAvailable": "????? ??????????",
29         "notAvailableTitle": "{token}? ?? {kind} ?????? ??? ? ?????? ??????????",
30         "notAvailableHelp": "????? ??????, ??????? ?????? ?????? ?????? ?????????? ? ??????
?????? ?????? ?????????? ??????? ????.",
31         "unreadable": "?????? ??????????? ? ?over 10s? ???? ?top 15 longest? ??????????.",
32         "hint": "10 ?????????????? ?????? ?????????????? over 10s ???? ???? ??? ? ???? ??????
????????? ?????? ????",
33         "try": "????? ??????????:",
34         "support": "<b>????? ????</b>? ?????????? ??????? ?????????? ??????????
?????????????. ???-????? <x>?????????</x> (????????????????).".
35     },
36     "grid": {
37         "empty": "??????? ???? ???? ???? ??????????",
38         "rally": "????? {n}",
39         "inReel": "????? {n}, ??????????",
40         "notInReel": "????? {n}, ??????????????",
41         "start": "?????????",
42         "long": "?????",
43         "shots": "~{count} ???????? · ??????????",
44         "shotsTitle": "????? ?????? ?????????????????? (?????????????????) ???
????????????????{conf}. ?????? ?????? ?????? ?????????? ??????? ????",
45         "shotsConfidence": " ? ??????: {conf}"
46     },
47     "footer": {
48         "count": "{count, plural, one {# ?????} other {# ?????????}}",
49         "summaryRest": "??????????? · ?????? {total}",
50         "reelName": "???? ??????",
51         "reelNamePlaceholder": "my-highlights",
52         "build": "???? ????? ({count})",
53         "building": "?????????????????",
54         "floorOne": "<b>????? ??? ? ?????????? ??????</b>? {min} ?????? ?????? ??????????
? ?????? ?????? ?????????????????, ?????????? ??? ? ??????????????. ?????? ?????????? ??????",
55         "floorAll": "<b>????????? ?????? {count} ??????????????</b> {min} ?????? ?????
????????? ? ?????? ?????????????? ?????????????????, ?????????? ??? ??????????????. ?????
????????????? ??????",
```


[illegible]

```

71      "uriPlaceholder": "gs://your-bucket/match.mp4",
72      "uriAriaLabel": "?????? ?? ????? ? ?? gs:// ???? URL ?? ???? ????? ?? ?? ???",
73      "hint": "?? gs:// ???? ?????, ?? ???? ??? ??? ???? ?? ?????? ?? ??? ????? ??????",
74      "submit": "?????? ??????? ?????",
75      "submitting": "???? ?? ??? ???",
76      "cancel": "???? ?????",
77      "working": "???? ?????? ??????? ?? ??? ???",
78      "progress": "{progress}",
79      "retry": "??? ?? ?????? ???"
80    },
81    "errors": {
82      "networkError": "???? ?? ????? ?????? ?? ???? ???? ?? ?? ??? ???",
83      "invalidUri": "???? ?????? ?? ????? ????? ? ?? gs:// ???? ?? ????? ?? ?? ?????",
84      "emptyResult": "?????????? ???? ??? ?? ?? ?????? ??? ??? ???? ???? ?????? ?? ???? ???
?????????? ??????????",
85      "timeout": "?? ??????? ?? ???? ??? ?? ??? ??? ??? ??? ?? ?? ??? ?? ???? ?? ? ??????
???? ?? ??? ?????? ???? , ?? ??? ?? ?????? ??????",
86      "engineGeneric": "??? ?????? ?? ??? ?????? ??? ?? ?????? ??????"
87    },
88    "selection": {
89      "count": "{count, plural, one {# ???? ?????} other {# ?????? ??????}}"
90    }
91  }
92

```

15. assistant (2026-06-21T15:13:56.625Z)

All three catalogs are consistent. Now let me read App.tsx onBuild and the build-error test, plus verify the i18n hygiene/parity test and check for any other `errorText`/`parseDetail` usages across the tree.

16. user (2026-06-21T15:13:57.577Z)

```

1 import { useEffect, useMemo, useState } from "react";
2 import { useTranslation, Trans } from "react-i18next";
3 import { useRallySelection } from "../hooks/useRallySelection";
4 import { RallyGrid } from "../components/RallyGrid";
5 import { QueryBar } from "../components/QueryBar";
6 import { SelectionFooter } from "../components/SelectionFooter";
7 import { LanguageSwitcher } from "../components/LanguageSwitcher";
8 import { CompileResult, type BuildState } from "../components/CompileResult";
9 import { IngestPanel } from "../components/IngestPanel";
10 import { runQuery, type ParsedQuery } from "../lib/query";
11 import { floorRisk } from "../lib/floor";
12 import { reelFilename } from "../lib/format";
13 import { classifyError } from "../lib/errors";
14 import { errorText } from "../lib/errorText";
15 import { log, newCorrelationId } from "../lib/log";
16 import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount }
from "../api/client";
17 import type { EngineConfig, RallySegment } from "../api/types";
18
19 /** One compact, structured description of how a query was understood (for the trace
log). */
20 function parseTrace(q: ParsedQuery) {
21   return {
22     raw: q.raw,
23     recognized: q.recognized,
24     filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
25     sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
26     limit: q.limit ?? null,
27     unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
28   };
29 }
30

```

```

31      // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
32      // contract; only honest fields are surfaced. A real session loads from POST /api/query.
33      const DEMO_RALLIES: RallySegment[] = [
34        { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
35        { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
36        { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
37        { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: { } },
38        { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
39        { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
40      ];
41
42      type LoadState =
43        | { status: "demo" }
44        | { status: "loading" }
45        | { status: "loaded"; count: number }
46        | { status: "error"; error: string };
47
48      export default function App() {
49        const { t } = useTranslation();
50        // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from
the URL,
51        // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
processed video
52        // without a reload (it also rewrites ?video= so a refresh is stable).
53        const [videoId, setVideoId] = useState<string | null>(
54          () => new URLSearchParams(window.location.search).get("video"),
55        );
56
57        const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
58        const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
59        const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
60        const [reelName, setReelName] = useState("highlights");
61        const [build, setBuild] = useState<BuildState>({ status: "idle" });
62
63        // Display order is separate from selection: a query can re-sort the cards while the
shared
64        // selection set (sel) stays the single source of truth for what lands in the reel.
65        const sel = useRallySelection(rallies, "all");
66
67        // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
68        useEffect(() => {
69          if (!videoId) return;
70          const cid = newCorrelationId();
71          log("info", "rallies.load_start", { cid, video_id: videoId });
72          // Show "loading" even when videoId was set dynamically by the ingest panel (initial
state was
73          // "demo" in that path) ? the banner must reflect the load, not stale demo copy.
74          setLoadState({ status: "loading" });
75          let alive = true;
76          listRallies(videoId, cid)
77            .then((rs) => {
78              if (!alive) return;
79              setRallies(rs);
80              setOrder(rs);
81              sel.apply(rs.map((r) => r.id));
82              setLoadState({ status: "loaded", count: rs.length });
83              log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });

```

```

84         })
85         .catch((err) => {
86             if (!alive) return;
87             setLoadState({ status: "error", error: String(err) });
88             log("error", "rallies.load_failed", { cid, video_id: videoId, error: String(err)
89         });
90         return () => {
91             alive = false;
92         };
93         // videoId is stable for the page; sel.apply is a stable callback.
94         // eslint-disable-next-line react-hooks/exhaustive-deps
95     }, [videoId]);
96
97     // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
98     // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
99     const [cfg, setCfg] = useState<EngineConfig | null>(null);
100     useEffect(() => {
101         let alive = true;
102         getConfig()
103             .then((c) => alive && setCfg(c))
104             .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
105         return () => {
106             alive = false;
107         };
108     }, []);
109     const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
110
111     // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
112     const ordinalOf = useMemo(() => {
113         const m = new Map<number, number>();
114         [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
115         return (id: number) => m.get(id) ?? 0;
116     }, [rallies]);
117
118     // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
119     const onBuild = async () => {
120         if (build.status === "building") return;
121         const cid = newCorrelationId();
122         const filename = reelFilename(reelName);
123         const ids = [...sel.selected];
124         if (!videoId) {
125             log("info", "build.no_video", { cid, selected: ids.length });
126             setBuild({
127                 status: "error",
128                 error: t("build.noVideo"),
129             });
130             return;
131         }
132         setBuild({ status: "building" });
133         log("info", "build.start", {
134             cid,
135             video_id: videoId,
136             filename,
137             selected: ids.length,
138             total_sec: Math.round(sel.totalSec),
139             below_floor: warning?.atRisk.length ?? 0,
140         });
141         try {
142             const res = await compileReel(videoId, ids, filename, cid);
143             const url = highlightUrl(videoId, filename);

```

```

144         setBuild({ status: "done", url, filename });
145         log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
146     } catch (err) {
147         // Same friendly mapping the ingest flow uses: a bare 5xx / network throw reads
"is the engine
148         // running?", a real engine 4xx surfaces its own detail ? never a raw "HTTP 500"
(A5).
149         const status = err instanceof ApiError ? err.status : undefined;
150         const e = classifyError(err);
151         setBuild({ status: "error", error: errorText(t, e) });
152         log("error", "build.failed", { cid, video_id: videoId, filename, status, code:
e.code, detail: e.detail, error: String(err) });
153     }
154 };
155
156     // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
157     // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
158     // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
159     const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
160         const cid = newCorrelationId();
161         log("info", "query.parse", { cid, source, ...parseTrace(q) });
162
163         // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
164         // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
165         if (q.unavailable.length > 0) {
166             log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
167         }
168
169         if (!q.recognized) {
170             log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
171             return; // empty / unparseable ? leave the user's curation untouched
172         }
173
174         try {
175             const { ordered, selectedIds } = runQuery(rallies, q);
176             setOrder(ordered);
177             sel.apply(selectedIds);
178             log("info", "query.apply", {
179                 cid,
180                 source,
181                 filters: q.filters.length,
182                 sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
183                 limit: q.limit ?? null,
184                 total: rallies.length,
185                 selected: selectedIds.length,
186                 selected_ids: selectedIds,
187                 dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
188             });
189             if (selectedIds.length === 0) {
190                 // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
191                 log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
192             }
193         } catch (err) {
194             // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
195             log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});

```

```

196     }
197   };
198
199   // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
200   const onToggle = (id: number) => {
201     log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
202     sel.toggle(id);
203   };
204   const onBulk = (op: "select_all" | "clear" | "invert") => {
205     const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
206     log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
207     if (op === "select_all") sel.selectAll();
208     else if (op === "clear") sel.clearAll();
209     else sel.invert();
210   };
211
212   return (
213     <main className="min-h-screen bg-paper text-ink pb-24">
214       <header className="bg-ink text-white">
215         <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
216           <span className="text-xl font-extrabold tracking-tight">
217             Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
218           </span>
219           <LanguageSwitcher />
220         </div>
221       </header>
222
223       <section className="mx-auto max-w-5xl px-6 py-10">
224         <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
225           {t("app.badge")}
226         </span>
227         <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
228         <p className="mt-2 max-w-2xl text-ink/70">
229           <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
230         </p>
231
232         {/* Front-half ingest (B-P2): when no video is loaded, point the tool at a
bucket URI / host
233           path ? process ? load. Once a video_id exists, the existing back-half takes
over. */}
234         {!videoId && (
235           <IngestPanel
236             onLoad={id => {
237               setVideoId(id);
238               window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
239             }}
240           />
241         )}
242
243         <ModeBanner load={loadState} videoId={videoId} />
244
245         <div className="mt-6">
246           <QueryBar onApply={onApplyQuery} />
247         </div>
248
249         <div className="mt-6 flex flex-wrap gap-2 text-sm">
250           <button type="button" onClick={() => onBulk("select_all")} className="inline-
flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-

```



```

green">{t("bulk.selectAll")}</button>
251         <button type="button" onClick={() => onBulk("clear")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.clear")}</button>
252         <button type="button" onClick={() => onBulk("invert")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.invert")}</button>
253     </div>
254
255     <div className="mt-5">
256         <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
257     </div>
258
259     <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
260 </section>
261
262 <SelectionFooter
263     count={sel.count}
264     totalSec={sel.totalSec}
265     reelName={reelName}
266     onReelNameChange={setReelName}
267     onBuild={onBuild}
268     building={build.status === "building"}
269     warning={warning}
270 />
271 </main>
272 );
273 }
274
275 function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
276     if (load.status === "demo") {
277         return (
278             <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
279                 <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
280             </p>
281         );
282     }
283     if (load.status === "loading") {
284         return (
285             <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
286                 <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code /> }} />
287             </p>
288         );
289     }
290     if (load.status === "error") {
291         return (
292             <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
text-red-800" data-testid="mode-banner">
293                 <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
components={{ code: <code /> }} />
294             </p>
295         );
296     }
297     return (
298         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
299             <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
<strong className="text-ink/70" /> }} />
300         </p>
301     );
302 }
303

```

```

1      // CI guardrails that keep i18n "by design" (FRIENDLY_AND_MULTILINGUAL.md §4):
2      // 1. KEY-PARITY ? every locale catalog has EXACTLY the `en` key set (the
extensibility proof:
3      //      drop a stub locale and this lists every gap). Trivially green while only `en`
exists; it
4      //      starts enforcing the moment a kn/hi catalog lands (B5/B6).
5      // 2. NO-BARE-STRINGS (attribute level, v1) ? user-facing component attributes
(placeholder /
6      //      aria-label / title) must route through t(), never a hardcoded literal. A curated
start per
7      //      the doc; an AST-level sweep of JSX text can extend it later.
8      import { describe, it, expect } from "vitest";
9      import { readdirSync, readFileSync } from "node:fs";
10     import { join } from "node:path";
11
12     const LOCALES_DIR = join(import.meta.dirname, "locales");
13     const SRC_DIR = join(import.meta.dirname, "..");
14
15     function flatten(obj: Record<string, unknown>, prefix = ""): string[] {
16         const keys: string[] = [];
17         for (const [k, v] of Object.entries(obj)) {
18             const key = prefix ? `${prefix}.${k}` : k;
19             if (v && typeof v === "object" && !Array.isArray(v)) keys.push(...flatten(v as
Record<string, unknown>, key));
20             else keys.push(key);
21         }
22         return keys.sort();
23     }
24
25     const loadKeys = (locale: string): string[] =>
26         flatten(JSON.parse(readFileSync(join(LOCALES_DIR, locale, "common.json"), "utf8")) as
Record<string, unknown>);
27
28     describe("i18n key-parity ? every locale has EXACTLY the en keys", () => {
29         const enKeys = loadKeys("en");
30         const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
31             .filter((d) => d.isDirectory() && d.name !== "en")
32             .map((d) => d.name);
33
34         it("en is the non-empty source catalog", () => {
35             expect(enKeys.length).toBeGreaterThan(20);
36         });
37
38         for (const loc of others) {
39             it(`${loc} matches en exactly (no missing, no extra keys)`, () => {
40                 const keys = loadKeys(loc);
41                 const missing = enKeys.filter((k) => !keys.includes(k));
42                 const extra = keys.filter((k) => !enKeys.includes(k));
43                 expect({ locale: loc, missing, extra }).toEqual({ locale: loc, missing: [], extra:
[] });
44             });
45         }
46     });
47
48     describe("no-bare-strings ? user-facing component attributes go through t()", () => {
49         const FILES = [
50             "App.tsx",
51             "components/QueryBar.tsx",
52             "components/RallyGrid.tsx",
53             "components/SelectionFooter.tsx",
54             "components/CompileResult.tsx",
55             "components/LanguageSwitcher.tsx",
56             "components/IngestPanel.tsx",
57         ];
58         // A hardcoded placeholder / aria-label / title with letters is a bare-string

```

```

regression ? it
59     // must be an expression ({t(...)}), not a quoted literal.
60     const BARE_ATTR = /\b(?:placeholder|aria-label|title)\s*=\s*"^[^"]*"|"[^"]*"|'[^']*'/g;
61
62     for (const f of FILES) {
63         it(`${f}: no hardcoded placeholder/aria-label/title`, () => {
64             const src = readFileSync(join(SRC_DIR, f), "utf8");
65             expect(src.match(BARE_ATTR) ?? []).toEqual([]);
66         });
67     }
68 });
69
70 describe("Indic fonts (B4) ? self-hosted OFL Noto, no runtime CDN", () => {
71     it("main.tsx bundles the OFL Noto Kannada + Devanagari fonts (self-hosted, not Google
Fonts)", () => {
72         const main = readFileSync(join(SRC_DIR, "main.tsx"), "utf8");
73         expect(main).toMatch(/@fontsource-variable\/noto-sans-kannada/);
74         expect(main).toMatch(/@fontsource-variable\/noto-sans-devanagari/);
75     });
76     it("the font stack falls back to Noto for Kannada + Devanagari glyphs", () => {
77         const css = readFileSync(join(SRC_DIR, "index.css"), "utf8");
78         expect(css).toMatch(/Noto Sans Kannada Variable/);
79         expect(css).toMatch(/Noto Sans Devanagari Variable/);
80     });
81 });
82
83 // The Studio SPA must render correctly on an AIR-GAPPED appliance (GPU/CPU/RAM, no
internet) ?
84 // every asset (fonts, styles, scripts) is bundled local, nothing pulled from a
CDN/Google Fonts.
85 describe("offline / self-contained ? no external runtime assets", () => {
86     const CDN = /googleapis|gstatic|fonts\.google|cdnjs|unpkg|jsdelivr\bcdn\.[a-z]/i;
87     const FILES = [
88         "../index.html", // web/index.html
89         "main.tsx",
90         "index.css",
91     ];
92     for (const rel of FILES) {
93         it(`${rel}: no external font/CDN reference`, () => {
94             expect(readFileSync(join(SRC_DIR, rel), "utf8")).not.toMatch(CDN);
95         });
96     }
97     it("web/index.html loads no absolute-URL <link>/<script src> (everything is
bundled/local)", () => {
98         const html = readFileSync(join(SRC_DIR, "..", "index.html"), "utf8");
99         expect(html).not.toMatch(/(?:href|src)\s*=\s*"?'https?:\/\//i);
100     });
101 });
102

```

18. user (2026-06-21T15:13:59.390Z)

```

web\src\App.tsx:13:import { classifyError } from "../lib/errors";
web\src\App.tsx:14:import { errorText } from "../lib/errorText";
web\src\App.tsx:150:     const e = classifyError(err);
web\src\App.tsx:151:     setBuild({ status: "error", error: errorText(t, e) });
web\src\hooks\useIngestJob.ts:11:import { classifyError, type IngestError } from
"../lib/errors";
web\src\hooks\useIngestJob.ts:83:         fail(classifyError(err), "ingest.poll_error");
web\src\hooks\useIngestJob.ts:122:        fail(classifyError(err), "ingest.submit_error");
web\src\components\IngestPanel.tsx:11:import { errorText } from "../lib/errorText";
web\src\components\IngestPanel.tsx:28: // Shared mapping with the build flow (lib/errorText):
`engine` failures carry the engine's own
web\src\components\IngestPanel.tsx:32: const errorMsg = phase.status === "error" ? errorText(t,
phase.error) : null;

```

```

web\src\components\build-error.test.tsx:4:// (classifyError ? errorText), so a non-technical
user never sees a raw "Network error calling POST
web\src\lib\errorText.test.ts:3:// errorText is the single mapping from a classified error ?
user-facing text, shared by the ingest
web\src\lib\errorText.test.ts:9:import { errorText } from "./errorText";
web\src\lib\errorText.test.ts:16:describe("errorText", () => {
web\src\lib\errorText.test.ts:18:    expect(errorText(i18n.t, { code: "engine", detail:
"min_shot_count too low" })).toBe("min_shot_count too low");
web\src\lib\errorText.test.ts:22:    const s = errorText(i18n.t, { code: "engine" });
web\src\lib\errorText.test.ts:29:    expect(errorText(i18n.t, { code: "networkError"
})).toMatch(/is it running/i);
web\src\lib\errorText.test.ts:34:    const s = errorText(i18n.t, { code });
web\src\lib\errorText.test.ts:42:    expect(errorText(i18n.t, { code: "networkError"
})).toBe("????? ??????????????. ?? ????");
web\src\lib\errorText.ts:11:export function errorText(t: TFunction, e: IngestError): string {
web\src\lib\errors.test.ts:2:import { parseDetail, classifyError } from "./errors";
web\src\lib\errors.test.ts:5:describe("parseDetail", () => {
web\src\lib\errors.test.ts:7:    expect(parseDetail(JSON.stringify({ detail: "unsupported
storage URI scheme" }))).toBe(
web\src\lib\errors.test.ts:12:    expect(parseDetail("plain text")).toBe("plain text");
web\src\lib\errors.test.ts:13:    expect(parseDetail(JSON.stringify({ other: 1
}))).toBe(JSON.stringify({ other: 1 }));
web\src\lib\errors.test.ts:14:    expect(parseDetail(JSON.stringify({ detail: { nested: true }
}))).toBe(JSON.stringify({ detail: { nested: true } }));
web\src\lib\errors.test.ts:18:describe("classifyError", () => {
web\src\lib\errors.test.ts:20:    expect(classifyError(new ApiError("net", { cid: "c"
}))).toEqual({ code: "networkError" });
web\src\lib\errors.test.ts:24:    expect(classifyError(err)).toEqual({ code: "engine", detail:
"unsupported storage URI scheme" });
web\src\lib\errors.test.ts:27:    expect(classifyError(new ApiError("x", { status: 500
}))).toEqual({ code: "networkError" });
web\src\lib\errors.test.ts:28:    expect(classifyError(new ApiError("x", { status: 502
}))).toEqual({ code: "networkError" });
web\src\lib\errors.test.ts:32:    expect(classifyError(err)).toEqual({ code: "engine", detail:
"failed to persist job record" });
web\src\lib\errors.test.ts:35:    expect(classifyError(new ApiError("x", { status: 404
}))).toEqual({ code: "engine", detail: "HTTP 404" });
web\src\lib\errors.test.ts:38:    expect(classifyError(new Error("boom"))).toEqual({ code:
"engine", detail: "Error: boom" });
web\src\lib\errors.ts:7:export function parseDetail(body: string): string {
web\src\lib\errors.ts:30:export function classifyError(err: unknown): IngestError {
web\src\lib\errors.ts:34:    const detail = parseDetail(err.body);

```

19. assistant (2026-06-21T15:14:03.724Z)

Now let me read the build-error test and the errorText test to verify the assertions, and the App.test if there is one, plus check the CompileResult for the "failed" copy reference.

20. user (2026-06-21T15:14:05.314Z)

```

1      // @vitest-environment jsdom
2      //
3      // A5 ? the BUILD path (App.onBuild ? CompileResult) now uses the SAME friendly mapping
as ingest
4      // (classifyError ? errorText), so a non-technical user never sees a raw "Network error
calling POST
5      // /api/compile" / "HTTP 500". A real engine 4xx still surfaces the engine's own detail
(the most
6      // specific, actionable message). Drives the real <App /> with a loaded video + a
stubbed engine.
7      import { describe, it, expect, beforeEach, afterEach, vi } from "vitest";
8      import { render, screen, cleanup, fireEvent } from "@testing-library/react";
9      import i18n from "../i18n";
10     import App from "../App";
11     import type { RallySegment } from "../api/types";

```

```

12
13     const RALLIES: RallySegment[] = [
14         { id: 1, start_time: 10, end_time: 20, duration: 10, server: null, receiver: null,
metadata: { shot_count: 12 } },
15         { id: 2, start_time: 30, end_time: 45, duration: 15, server: null, receiver: null,
metadata: { shot_count: 14 } },
16     ];
17
18     const httpOk = (status: number, json: unknown) => ({
19         ok: status >= 200 && status < 300, status, json: async () => json, text: async () =>
JSON.stringify(json),
20     });
21     const httpErr = (status: number, body: string) => ({
22         ok: false, status, json: async () => JSON.parse(body), text: async () => body,
23     });
24
25     // /api/query ? rallies; /api/config ? 500 (cfg unavailable ? no floor warning);
/api/compile ? caller-supplied.
26     function mockFetch(compile: () => unknown) {
27         return vi.fn(async (url: string | URL, init?: RequestInit) => {
28             const u = String(url);
29             const method = init?.method ?? "GET";
30             if (u === "/api/query" && method === "POST") return httpOk(200, { play_segments:
RALLIES });
31             if (u === "/api/config") return httpErr(500, "");
32             if (u === "/api/compile" && method === "POST") return compile();
33             throw new Error(`unexpected fetch ${method} ${u}`);
34         }) as unknown as typeof fetch;
35     }
36
37     beforeEach(async () => {
38         window.history.pushState({}, "", "?video=vidX"); // job-tethered: load a real video's
rallies
39         window.localStorage.clear();
40         await i18n.changeLanguage("en");
41     });
42
43     afterEach(() => {
44         cleanup();
45         vi.unstubAllGlobals();
46         window.history.pushState({}, "", "/");
47     });
48
49     // Rallies load all-selected (count 2) ? the Build button label proves the load
completed.
50     const loadAndBuild = async () => {
51         render(<App />);
52         fireEvent.click(await screen.findByRole("button", { name: /build reel \d/i }));
53     };
54
55     describe("A5: friendly build-error mapping", () => {
56         it("a network / engine-down compile failure shows the friendly 'is it running?' line",
async () => {
57             vi.stubGlobal("fetch", mockFetch(() => {
58                 throw new Error("offline");
59             }));
60             await loadAndBuild();
61             expect(await screen.findByText(/can't reach khelsutra/i)).toBeTruthy();
62             expect(screen.queryByText(/network error calling/i)).toBeNull(); // the raw client
message is gone
63         });
64
65         it("a real engine 4xx surfaces the engine's own detail verbatim", async () => {
66             vi.stubGlobal("fetch", mockFetch(async () => httpErr(400, JSON.stringify({ detail:
"rally below min_shot_count" })))));

```

```

67         await loadAndBuild();
68         expect(await screen.findByText(/rally below min_shot_count/i)).toBeTruthy();
69     });
70 });
71

```

21. user (2026-06-21T15:14:05.619Z)

```

1      // @vitest-environment jsdom
2      //
3      // errorText is the single mapping from a classified error ? user-facing text, shared by
the ingest
4      // and build flows (A5). These pin: the engine message passes through verbatim, a
detail-less engine
5      // error degrades to a friendly localized line (never a raw key), and every non-engine
code resolves
6      // to a real catalog string in the shared `errors.*` namespace.
7      import { describe, it, expect, beforeAll } from "vitest";
8      import i18n from "../i18n";
9      import { errorText } from "../errorText";
10     import type { IngestErrorCode } from "../errors";
11
12     beforeAll(async () => {
13         await i18n.changeLanguage("en");
14     });
15
16     describe("errorText", () => {
17         it("an engine error shows the engine's own detail verbatim (most specific message)",
() => {
18             expect(errorText(i18n.t, { code: "engine", detail: "min_shot_count too low"
})).toBe("min_shot_count too low");
19         });
20
21         it("an engine error with no detail degrades to a friendly localized line (not a raw
key)", () => {
22             const s = errorText(i18n.t, { code: "engine" });
23             expect(s).toBe(i18n.t("errors.engineGeneric"));
24             expect(s).not.toMatch(/^errors\.\/);
25             expect(s.length).toBeGreaterThan(0);
26         });
27
28         it("networkError maps to the friendly 'is it running?' line", () => {
29             expect(errorText(i18n.t, { code: "networkError" })).toMatch(/is it running/i);
30         });
31
32         it("every non-engine code resolves to a real catalog string (never a raw key)", () =>
{
33             for (const code of ["invalidUri", "networkError", "emptyResult", "timeout"] as
IngestErrorCode[]) {
34                 const s = errorText(i18n.t, { code });
35                 expect(s, code).not.toMatch(/^errors\.\/);
36                 expect(s.length, code).toBeGreaterThan(0);
37             }
38         });
39
40         it("localizes per the active locale (Kannada network message)", async () => {
41             await i18n.changeLanguage("kn");
42             expect(errorText(i18n.t, { code: "networkError" })).toBe("?????? ????????????????
??? ??????????????????");
43             await i18n.changeLanguage("en");
44         });
45     });
46

```

22. user (2026-06-21T15:14:05.858Z)

```

1      // Typed client for the engine FastAPI. SAME-ORIGIN relative `/api/*` (the appliance
serves
2      // UI + engine behind one origin; in dev Vite proxies `/api` ? :8000). Every call
carries a
3      // correlation id (header + logs) so a journey is traceable end to end.
4      import { log, newCorrelationId } from "../lib/log";
5      import type { RallySegment, QueryResponse, CompileResponse, EngineConfig,
ProcessResponse, JobStatus } from "../types";
6
7      const BASE = "/api";
8
9      export class ApiError extends Error {
10         readonly cid?: string;
11         readonly status?: number;
12         readonly body?: string;
13         constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
14             super(message);
15             this.name = "ApiError";
16             this.cid = opts.cid;
17             this.status = opts.status;
18             this.body = opts.body;
19             if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
20         }
21     }
22
23     interface RequestOpts {
24         method?: string;
25         body?: string;
26         headers?: Record<string, string>;
27         /** Correlation id to thread through this call; auto-generated if absent. */
28         cid?: string;
29     }
30
31     async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
32         const cid = opts.cid ?? newCorrelationId();
33         const url = `${BASE}${path}`;
34         const method = opts.method ?? "GET";
35         log("info", "api.request", { cid, method, url });
36
37         let res: Response;
38         try {
39             res = await fetch(url, {
40                 method,
41                 body: opts.body,
42                 headers: { "content-type": "application/json", "x-correlation-id": cid,
...(opts.headers ?? {}) },
43             });
44         } catch (err) {
45             // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
46             log("error", "api.network_error", { cid, method, url, error: String(err) });
47             throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
48         }
49
50         if (!res.ok) {
51             const body = await res.text().catch(() => "");
52             log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
53             throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
54         }
55
56         const data = (await res.json()) as T;
57         log("info", "api.response", { cid, method, url, status: res.status });
58         return data;

```

```

59     }
60
61     /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
62     * processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
fails 400
63     * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
64     export async function processVideo(videoPath: string, cid?: string):
Promise<ProcessResponse> {
65         return request<ProcessResponse>("/process", {
66             method: "POST",
67             body: JSON.stringify({ video_path: videoPath }),
68             cid,
69         });
70     }
71
72     /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
73     export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
74         return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
75     }
76
77     /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
78     export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
79         const data = await request<QueryResponse>("/query", {
80             method: "POST",
81             body: JSON.stringify({ video_id: videoId }),
82             cid,
83         });
84         return data.play_segments ?? [];
85     }
86
87     /** Stitch selected rallies into one reel. POST /api/compile {video_id, segment_ids,
output_filename} [main.py:342]. */
88     export async function compileReel(
89         videoId: string,
90         segmentIds: number[],
91         outputFilename: string,
92         cid?: string,
93     ): Promise<CompileResponse> {
94         return request<CompileResponse>("/compile", {
95             method: "POST",
96             body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename }),
97             cid,
98         });
99     }
100
101     /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
102     export function highlightUrl(videoId: string, filename: string): string {
103         return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
104     }
105
106     /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
107     export async function getConfig(cid?: string): Promise<EngineConfig> {
108         return request<EngineConfig>("/config", { method: "GET", cid });
109     }
110
111     /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */

```



```

112     export function minShotCount(cfg: EngineConfig): number | undefined {
113         return cfg.stitching?.min_shot_count;
114     }
115

```

23. assistant (2026-06-21T15:14:11.015Z)

Now I have a complete picture. Let me verify the build path mapping logic carefully and run the test suite to confirm green, plus run a final grep for any `ingest.failed` reference (the `result.failed` key still exists ? that's distinct). Let me also confirm the `useIngestJob` hook uses `timeout`/`emptyResult`/`invalidUri` codes that map to `errors.\*`.

24. user (2026-06-21T15:14:12.450Z)

```

1      // Drives the front-half ingest: submit a video to POST /api/process, then poll GET
/api/jobs/{id}
2      // until it finishes, and hand the resulting video_id to the existing rally-loading
back-half.
3      //
4      // Kept i18n-FREE (returns stable IngestErrorCodes, not prose) so it unit-tests without
a
5      // translation context ? IngestPanel maps codes to localized strings. The poll loop is a
RECURSIVE
6      // setTimeout (never setInterval): each tick waits for the prior getJob to resolve, so a
slow engine
7      // can't pile up overlapping requests. It backs off, gives up after a ceiling, and is
fully
8      // cancellable on unmount / user-cancel / a superseding submit.
9      import { useCallback, useEffect, useRef, useState } from "react";
10     import { getJob, processVideo } from "../api/client";
11     import { classifyError, type IngestError } from "../lib/errors";
12     import { log, newCorrelationId } from "../lib/log";
13     import type { JobStatus } from "../api/types";
14
15     const POLL_MIN_MS = 1000;
16     const POLL_MAX_MS = 5000;
17     const POLL_BACKOFF = 1.5;
18     const POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' job won't poll forever
19
20     export type IngestPhase =
21     | { status: "idle" }
22     | { status: "submitting" }
23     | { status: "polling"; progress: string | null }
24     | { status: "done"; videoId: string }
25     | { status: "error"; error: IngestError };
26
27     interface Run {
28         cancelled: boolean;
29         timer?: ReturnType<typeof setTimeout>;
30     }
31
32     export function useIngestJob(onLoaded?: (videoId: string) => void) {
33         const [phase, setPhase] = useState<IngestPhase>({ status: "idle" });
34         const runRef = useRef<Run | null>(null);
35         // Keep the latest onLoaded without making submit depend on it (a parent re-render
won't restart polls).
36         const onLoadedRef = useRef(onLoaded);
37         onLoadedRef.current = onLoaded;
38
39         const stop = useCallback(() => {
40             const run = runRef.current;
41             if (run) {
42                 run.cancelled = true;
43                 if (run.timer) clearTimeout(run.timer);
44

```

```

45     runRef.current = null;
46   }, []);
47
48   // Cancel any in-flight poll loop when the component unmounts (App.tsx:80-82 alive-
49   // flag idiom).
50   useEffect(() => stop, [stop]);
51
52   const submit = useCallback(
53     (rawUri: string) => {
54       const uri = rawUri.trim();
55       if (!uri) {
56         setPhase({ status: "error", error: { code: "invalidUri" } });
57         return;
58       }
59
60       stop(); // supersede any prior run
61       const run: Run = { cancelled: false };
62       runRef.current = run;
63
64       const cid = newCorrelationId();
65       const startedAt = Date.now();
66       setPhase({ status: "submitting" });
67       log("info", "ingest.submit_start", { cid, uri });
68
69       const fail = (error: IngestError, event: string) => {
70         if (run.cancelled) return;
71         setPhase({ status: "error", error });
72         const level = error.code === "timeout" || error.code === "emptyResult" ? "warn"
73         : "error";
74         log(level, event, { cid, uri, code: error.code, detail: error.detail });
75       };
76
77       const poll = (jobId: string, delay: number) => {
78         run.timer = setTimeout(() => {
79           void (async () => {
80             if (run.cancelled) return;
81             let job: JobStatus;
82             try {
83               job = await getJob(jobId, cid);
84             } catch (err) {
85               fail(classifyError(err), "ingest.poll_error");
86               return;
87             }
88             if (run.cancelled) return;
89
90             if (job.status === "done") {
91               const videoId = job.video_id ?? job.result?.video_id ?? null;
92               // Two status layers: a `done` job with a `failed` pipeline verdict (or no
93               // video_id) is
94               // a success-shaped FAILURE ? never load it, never hang.
95               if (job.result?.status === "failed" || !videoId) {
96                 fail({ code: "emptyResult" }, "ingest.empty_result");
97                 return;
98               }
99               setPhase({ status: "done", videoId });
100               log("info", "ingest.done", { cid, uri, video_id: videoId });
101               onLoadedRef.current?.(videoId);
102               return;
103             }
104             if (job.status === "failed") {
105               fail({ code: "engine", detail: job.error ?? "job failed" },
106                 "ingest.job_failed");
107             }
108             return;
109           })
110         }, delay);
111       };
112
113       // queued | running ? reflect coarse progress (free-text phase) and keep

```

```

polling.
106         setPhase({ status: "polling", progress: job.progress ?? null });
107         if (Date.now() - startedAt > POLL_CEILING_MS) {
108             fail({ code: "timeout" }, "ingest.poll_timeout");
109             return;
110         }
111         poll(jobId, Math.min(delay * POLL_BACKOFF, POLL_MAX_MS));
112     })();
113 }, delay);
114 };
115
116 void (async () => {
117     let jobId: string;
118     try {
119         const resp = await processVideo(uri, cid);
120         jobId = resp.job_id;
121     } catch (err) {
122         fail(classifyError(err), "ingest.submit_error");
123         return;
124     }
125     if (run.cancelled) return;
126     log("info", "ingest.queued", { cid, uri, job_id: jobId });
127     setPhase({ status: "polling", progress: null });
128     poll(jobId, POLL_MIN_MS);
129 })();
130 },
131 [stop],
132 );
133
134 const cancel = useCallback(() => {
135     stop();
136     setPhase({ status: "idle" });
137     log("info", "ingest.cancel", {});
138 }, [stop]);
139
140 const reset = useCallback(() => {
141     stop();
142     setPhase({ status: "idle" });
143 }, [stop]);
144
145 const busy = phase.status === "submitting" || phase.status === "polling";
146 return { phase, submit, cancel, reset, busy };
147 }
148

```

25. user (2026-06-21T15:14:13.378Z)

No matches found

Found 0 total occurrences across 0 files.

26. assistant (2026-06-21T15:14:17.328Z)

The grep for `errors.\*` in JSON returned 0 ? that's because the keys are JSON keys (`"networkError":`), not `errors.networkError` literals. I already confirmed their presence by reading all three catalogs directly. Let me run the test suite to verify green, then finalize.

27. user (2026-06-21T15:14:25.572Z)

```

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) >
B5: switching to Kannada renders the kn catalog (Kannada copy, not English)
{"ts":"2026-06-21T15:14:25.051Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

```

```

? src/components/build-error.test.tsx (2 tests)
595ms
? A5: friendly build-error mapping > a network / engine-down
compile failure shows the friendly 'is it running?' line 468ms
stdout | src/i18n/i18n.test.tsx > i18n scaffold (B1) > B6:
switching to Hindi renders the hi catalog (extensibility proof ? just a catalog)
{"ts":"2026-06-21T15:14:25.076Z","level":"info","event":"api.request","cid":"dfa42e59-6693-4aee-aed3-fa6be3295075","method":"GET","url":"/api/config"}

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) >
B6: switching to Hindi renders the hi catalog (extensibility proof ? just a catalog)
{"ts":"2026-06-21T15:14:25.111Z","level":"error","event":"api.network_error","cid":"dfa42e59-6693-4aee-aed3-fa6be3295075","method":"GET","url":"/api/config","error":"Error: no engine in test"}

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) >
B6: switching to Hindi renders the hi catalog (extensibility proof ? just a catalog)
{"ts":"2026-06-21T15:14:25.111Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

stdout | src/i18n/i18n.test.tsx > i18n scaffold (B1) > a
locale with no catalog falls back to English (never a raw key)
{"ts":"2026-06-21T15:14:25.130Z","level":"info","event":"api.request","cid":"a9835595-e6fb-4c15-ab02-98e05bc63a4b","method":"GET","url":"/api/config"}

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) >
a locale with no catalog falls back to English (never a raw key)
{"ts":"2026-06-21T15:14:25.134Z","level":"error","event":"api.network_error","cid":"a9835595-e6fb-4c15-ab02-98e05bc63a4b","method":"GET","url":"/api/config","error":"Error: no engine in test"}

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) >
a locale with no catalog falls back to English (never a raw key)
{"ts":"2026-06-21T15:14:25.134Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

stdout | src/i18n/i18n.test.tsx > browser-language negotiation (B1:
N1/N3?N7) > N6: an unsupported browser language (Tamil) serves English content ?
never a locale we can't render
{"ts":"2026-06-21T15:14:25.150Z","level":"info","event":"api.request","cid":"c2d9ff41-0676-43f6-840d-03fba4a5e938","method":"GET","url":"/api/config"}

stderr | src/i18n/i18n.test.tsx > browser-language negotiation (B1:
N1/N3?N7) > N6: an unsupported browser language (Tamil) serves English content ?
never a locale we can't render
{"ts":"2026-06-21T15:14:25.153Z","level":"error","event":"api.network_error","cid":"c2d9ff41-0676-43f6-840d-03fba4a5e938","method":"GET","url":"/api/config","error":"Error: no engine in test"}

stderr | src/i18n/i18n.test.tsx > browser-language negotiation (B1:
N1/N3?N7) > N6: an unsupported browser language (Tamil) serves English content ?
never a locale we can't render
{"ts":"2026-06-21T15:14:25.153Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

? src/i18n/i18n.test.tsx (13 tests)
693ms
? i18n scaffold (B1) > renders user-facing copy from the en
catalog (not raw keys) 389ms

```

Test Files 14 passed (14)
Tests 135 passed (135)
Start at 20:44:20
Duration 4.21s (transform 882ms, setup 272ms, collect 4.00s, tests 2.47s,
environment 13.50s, prepare 4.02s)

28. assistant (2026-06-21T15:14:29.933Z)

All 135 tests pass. Let me do one final confirmation grep for ``t("ingest.failed")`` specifically and any remaining usage of the removed keys across the whole repo (including site/ and any non-src locations), and verify ``ingest.failed`` is truly gone from catalogs (the ``result.failed`` key is distinct).

29. user (2026-06-21T15:14:32.199Z)

```
web\src\i18n\locales\kn\common.json:61: "failed": "???? ??????????",
web\src\i18n\locales\en\common.json:61: "failed": "Build failed",
web\src\i18n\locales\hi\common.json:61: "failed": "????? ?????",
```

30. user (2026-06-21T15:14:33.286Z)

No matches found

31. assistant (2026-06-21T15:14:50.572Z)

The only ``"failed"`` keys remaining are under ``result.*`` (line 61, the "Build failed" header) ? that is a distinct, still-used key, not ``ingest.failed``. No ``ingest.failed``/``ingest.networkError``/etc. references exist anywhere in the repo (excluding `node_modules``).

All four checks are clean. Let me record my findings.

The review is complete. All four checks pass cleanly:

****1. Orphaned keys**** ? Grep across the entire ``web/src`` tree (and the whole repo excluding `node_modules``) for ``t("ingest.failed")``, ``t("ingest.networkError")``, ``t("ingest.invalidUri")``, ``t("ingest.emptyResult")``, ``t("ingest.timeout")``, and template ``t(`ingest.${...}`)`` returned zero matches. The only surviving ``"failed"`` key is ``result.failed`` (en/kn/hi line 61, "Build failed" header ? used by `CompileResult``), which is distinct and intentional. The hook (``useIngestJob.ts``) emits codes (``invalidUri``/``emptyResult``/``timeout``/``engine``/``networkError``) that ``errorText`` maps to the new ``errors.*`` namespace; no raw-key render path remains.

****2. Key-parity**** ? ``errors.{networkError,invalidUri,emptyResult,timeout,engineGeneric}`` are present and identical in structure across all three catalogs (en/kn/hi lines 81-87). The removed ``ingest.{networkError,invalidUri,emptyResult,timeout}`` and ``ingest.failed`` are gone from all three; ``ingest.*`` retains exactly `title/intro/uriPlaceholder/uriAriaLabel/hint/submit/submitted/cancel/working/progress/retry``. No asymmetry ? the hygiene key-parity test (``hygiene.test.ts``) stays green.

****3. Build mapping correctness**** ? ``classifyError`` (errors.ts:30-44) + ``App.onBuild`` (App.tsx:146-153): a bare 5xx with no body ? ``networkError`` ? friendly "is it running?" line (errors.ts:40); any HTTP error WITH a parseable ``{detail}`` ? ``engine`` with verbatim detail (errors.ts:33-35); a fetch throw (`ApiError``, ``status===undefined``) ? ``networkError`` (errors.ts:32). No case shows a worse message than before ? the previous build path used ``parseDetail(err.body) ?? String(err)``, which for a bare 5xx/network throw would surface the raw "Network error calling POST /api/compile" or "HTTP 500"; the new path replaces only those with a friendly line while still surfacing a real engine detail verbatim. A 4xx with no body now reads ``HTTP {status}`` (errors.ts:41) instead of the raw client message ? equivalent or better.

****4. Ingest test text**** ? ``IngestPanel.test.tsx``: the 400-detail assertion (``/unsupported storage URI scheme/i``, line 80) flows through ``classifyError`` ? ``{code:"engine", detail}`` ? ``errorText`` returns the detail verbatim. The unreachable-engine assertion (``/can't reach khelsutra/i``, line 90) resolves to ``errors.networkError`` = "Can't reach KhelSutra. Is it running?" (en line 82) ? the rendered text is unchanged from the old inline

``ingest.networkError`` copy. Both produce identical strings under the new ``errors.*`` copy.

Test suite: 135/135 green, including ``errorText.test.ts``, ``build-error.test.tsx``, ``IngestPanel.test.tsx``, and the i18n key-parity hygiene test.

32. user (2026-06-21T15:14:55.602Z)

Output does not match required schema: root: must have required property 'findings'

33. user (2026-06-21T15:15:03.704Z)

Output does not match required schema: root: must have required property 'findings'

34. assistant (2026-06-21T15:15:07.006Z)

The ``findings`` field needs to be an actual array, not a string. Let me call it with the proper structure.

35. user (2026-06-21T15:15:11.543Z)

Output does not match required schema: root: must have required property 'findings'

36. assistant (2026-06-21T15:15:15.912Z)

The empty findings array is being dropped. Let me include one informational finding (nit severity) so the array is non-empty and clearly serializes.

37. user (2026-06-21T15:15:24.937Z)

Output does not match required schema: root: must have required property 'findings'

38. assistant (2026-06-21T15:15:28.914Z)

The error persists even with a valid non-empty array, which suggests the JSON string for ``findings`` isn't being parsed into an array. Let me try passing findings as a properly-formed array without the string wrapper ambiguity ? providing each field directly.

39. user (2026-06-21T15:15:36.134Z)

Output does not match required schema: root: must have required property 'findings'

40. assistant (2026-06-21T15:15:40.810Z)

The validator reports ``findings`` missing at root regardless of value, which means the parameter isn't being recognized ? likely the large ``summary`` plus ``findings`` together are tripping something, or the harness expects both as top-level keys and one is being lost. Let me try a minimal call with a short summary and a minimal findings array to isolate.

41. user (2026-06-21T15:15:42.164Z)

Output does not match required schema: root: must have required property 'findings'

42. assistant (2026-06-21T15:15:47.977Z)

Still failing even minimally. The issue is that ``findings`` is being passed as a JSON-encoded string rather than a native array, and the schema requires an actual array property. The harness may be double-encoding. Let me try passing an empty array as the literal value ``[]`` to see if the array type itself is the problem, with findings as the only thing varying.

43. user (2026-06-21T15:15:48.760Z)

Output does not match required schema: root: must have required property 'findings'

44. assistant (2026-06-21T15:15:53.787Z)

The schema requires `findings` but it's consistently reported missing ? this is a serialization quirk where the array argument isn't reaching the validator. Let me check if the `required` list is `["summary", "findings"]` and try giving findings a clearly-typed object array using a single complete object (all required subfields: severity, file, title, detail).

45. user (2026-06-21T15:16:04.074Z)

Output does not match required schema: root: must have required property 'findings'

46. assistant (2026-06-21T15:16:18.663Z)

The validator consistently rejects every form, even valid ones. The `findings` array argument is not being delivered to the validator ? this appears to be a harness serialization issue with how I'm encoding the array. Let me try expressing both parameters in a way that avoids any ambiguity: a very minimal payload.

47. user (2026-06-21T15:16:18.862Z)

Structured output provided successfully

48. assistant (2026-06-21T15:16:28.714Z)

The minimal call succeeded ? the issue was payload size/encoding in the longer summary. Now let me submit the full, correct findings with a properly sized summary.

49. user (2026-06-21T15:16:35.865Z)

Output does not match required schema: root: must have required property 'findings'

50. assistant (2026-06-21T15:16:38.853Z)

Confirmed: the longer summary is what breaks delivery of `findings`. The minimal version worked. Let me keep the summary compact and findings intact.

51. user (2026-06-21T15:16:43.382Z)

Output does not match required schema: root: must have required property 'findings'

52. assistant (2026-06-21T15:16:47.152Z)

Still failing with the findings object present. The earlier success had empty findings `[]`. The problem is the non-empty findings array. Let me test whether a non-empty findings array works at all with a minimal summary.

53. user (2026-06-21T15:16:47.965Z)

Structured output provided successfully

54. assistant (2026-06-21T15:16:59.251Z)

That succeeded. The earlier failures were caused by special characters in my long summary (the apostrophe-stripped "cant", em-dashes, slashes, or parentheses combinations) corrupting the argument encoding. The structured output is now delivered successfully with the full nit finding included.

To summarize the review for the record:

****A5 refactor reviewed ? clean. No regressions, no orphaned i18n keys. 135/135 tests green.****

1. ****Orphaned keys**** ? Zero references to the removed
`ingest.{failed,networkError,invalidUri,emptyResult,timeout}` keys or the `t(\`ingest.\${...}\`)

template form anywhere in `web/src` or the wider repo. The only surviving `failed` key is the distinct, still-used `result.failed` ("Build failed" header).

2. **Key-parity** ? `errors.{networkError,invalidUri,emptyResult,timeout,engineGeneric}` present and identical in all three catalogs (en/kn/hi, lines 81-87); removed ingest keys gone from all three; `ingest.\*` retains exactly the kept set. No asymmetry; hygiene key-parity test stays green.

3. **Build mapping** ? `classifyError` (errors.ts:30-44) + `App.onBuild` (App.tsx:146-153): bare 5xx and fetch-throw ? `networkError` (friendly line); any HTTP error with `{detail}` ? `engine` (verbatim). Strictly improves the prior `parseDetail(err.body) ?? String(err)` for the bare-5xx/network case; no path is worse.

4. **Ingest test text** ? "can't reach khelsutra" resolves to `errors.networkError` (en:82); the 400 "unsupported storage URI scheme" detail passes through `engine` verbatim. Same rendered strings as before.

One nit (out of scope): `parseDetail` in `web/src/lib/errors.ts` is now consumed only internally (errors.ts:34 + its own test) but is still exported ? harmless, not a regression.